

ASSIGNMENT-01

Name: Kaveti Sreenavya

HT.No: 2303A51244

Batch: 19

Task 1: AI-Generated Logic Without Modularization (Fibonacci Sequence Without Functions)

Scenario

You are asked to write a quick numerical sequence generator for a learning platform prototype.

Prompt: #Fibonacci series upto n terms

Code:

```

1 #Task 1: AI-Generated Logic Without Modularization (Fibonacci Sequence Without Functions)
2 #fibonacci series up to n terms
3 # Accept user input
4
5 n = int(input("Enter the number of terms: "))
6
7 # Initialize first two terms
8 a = 0
9 b = 1
10
11 # Handle edge cases
12 if n <= 0:
13     print("Please enter a positive integer")
14 elif n == 1:
15     print(a)
16 else:
17     print(a, b, end=" ")
18     for i in range(2, n):
19         c = a + b
20         print(c, end=" ")
21         a = b
22         b = c

```

Result:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python +v [] ... [] [] []
● vaishnavibairagoni@Vaishnavis-Laptop AI AC % python3 assign1.py

Enter the number of terms: 15
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377

● vaishnavibairagoni@Vaishnavis-Laptop AI AC % python3 assign1.py

Enter the number of terms: 15
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
○ vaishnavibairagoni@Vaishnavis-Laptop AI AC %
○ vaishnavibairagoni@Vaishnavis-Laptop AI AC %
○ vaishnavibairagoni@Vaishnavis-Laptop AI AC % python3 assign1.py

Enter the number of terms: 15
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 Enter the number of terms: 15
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 Enter the number of terms:
Indexing completed.
```

Observation:

Task 2: AI Code Optimization & Cleanup (Improving Efficiency)

Scenario

The prototype will be shared with other developers and needs optimization.

Prompt: Optimize this code

Code:

```
assign1.py X
assign1.py > ...
24 #Task 2: AI Code Optimization & Cleanup (Improving Efficiency)
25 n = int(input("Enter the number of terms: "))
26
27 prev, curr = 0, 1
28
29 for i in range(n):
30     print(prev, end=" ")
31     prev, curr = curr, prev + curr
32
```

Result:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
vaishnavibairagoni@Vaishnavis-Laptop AI AC % python3 assign1.py
Enter the number of terms: 15
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
vaishnavibairagoni@Vaishnavis-Laptop AI AC % python3 assign1.py
Enter the number of terms: 15
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
vaishnavibairagoni@Vaishnavis-Laptop AI AC %
vaishnavibairagoni@Vaishnavis-Laptop AI AC %
vaishnavibairagoni@Vaishnavis-Laptop AI AC % python3 assign1.py
Enter the number of terms: 15
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 Enter the number of terms: 15
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 Enter the number of terms:
Indexing completed.
Ln 9, Col 6 Spaces: 4 UTF-8 LF Python 3.14.2 PGO Go Live
```

Observation:

Scenario

Prompt: Optimize this code using functions

Code:

```
33 #Task 3: Modular Design Using AI Assistance (Fibonacci Using Functions)
34 # Function to generate Fibonacci sequence up to n terms
35 def fibonacci(n):
36     sequence = []
37     prev, curr = 0, 1
38
39     for i in range(n):
40         sequence.append(prev)
41         prev, curr = curr, prev + curr
42
43     return sequence
44
45
46 # Take user input
47 n = int(input("Enter the number of terms: "))
48
49 # Call the function and print the result
50 result = fibonacci(n)
51 print("Fibonacci sequence:", result)
52
```

Result:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + v [ ] [ ] ... [ ]
```

```
● vaishnavibairagoni@Vaishnavis-Laptop AI AC % python3 assign1.py  
  
Enter the number of terms: 15  
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377  
● vaishnavibairagoni@Vaishnavis-Laptop AI AC % python3 assign1.py  
  
Enter the number of terms: 15  
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 [ ]  
○ vaishnavibairagoni@Vaishnavis-Laptop AI AC %  
○ vaishnavibairagoni@Vaishnavis-Laptop AI AC %  
○ vaishnavibairagoni@Vaishnavis-Laptop AI AC % python3 assign1.py  
  
Enter the number of terms: 15  
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 Enter the number of terms: 15  
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 Enter the number of terms: [ ]
```

```
Indexing completed.
```

```
Ln 9, Col 6 Spaces: 4 UTF-8 LF [ ] Python P3 3.14.2 B-G Live
```

Observation:

Task 4: Comparative Analysis – Procedural vs Modular Fibonacci Code

Scenario

You are participating in a code review session

Prompt: Description on comparison between with functions and without functions

Criteria	Without Functions (Task 1)	With Functions (Task 3)	
Code Clarity	Logic is mixed with input/output, making it harder to read	Logic is isolated inside a function, easier to understand	
Reusability	Cannot be reused without copying the code	Function can be reused across multiple files or modules	
Debugging Ease	Harder to debug because everything runs in one block	Easier to debug by testing the function independently	
Maintainability	Changes require editing the entire script	Changes are limited to the function	
Suitability for Large Systems	Not suitable; leads to duplicated logic	Highly suitable; supports modular design	
Testing	Manual testing only	Function can be unit tested easily	
Scalability	Poor scalability	Good scalability	

Observation:

The non-function-based Fibonacci program is suitable only for small, one-time scripts or learning exercises. Since the logic is written directly in the main code, it lacks reusability and becomes difficult to maintain or debug as the application grows.

The function-based implementation significantly improves code quality. By separating the Fibonacci logic into a user-defined function, the program becomes modular, reusable, and easier to test. This approach aligns with best practices in software development and is more appropriate for larger systems where the same logic may be required in multiple modules.

Task 5: AI-Generated Iterative vs Recursive Fibonacci Approaches (Different Algorithmic Approaches for Fibonacci Series)

Scenario

Your mentor wants to assess AI's understanding of different algorithmic paradigms.

Code:

```
assign1.py X
assign1.py > ...
54 # Task 5: AI-Generated Iterative vs Recursive Fibonacci Approaches (Different Algorithmic Approaches for Fibonacci Series)
55 # Iterative approach
56 def fibonacci_iterative(n):
57     prev, curr = 0, 1
58     result = []
59
60     for i in range(n):
61         result.append(prev)
62         prev, curr = curr, prev + curr
63
64     return result
65
66
67 # User input
68 n = int(input("Enter number of terms: "))
69 print("Iterative Fibonacci:", fibonacci_iterative(n))
70
71
72 # Recursive approach
73 def fibonacci_recursive(n):
74     if n <= 1:
75         return n
76     return fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2)
77
78
79 # User input
80 n = int(input("Enter number of terms: "))
81
82 print("Recursive Fibonacci:")
83 for i in range(n):
84     print(fibonacci_recursive(i), end=" ")
```

Result:

```
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 Enter the number of terms: 15
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 Enter the number of terms: 15
Fibonacci sequence: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377]
Enter number of terms: 15
Iterative Fibonacci: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377]
Enter number of terms: 15
Recursive Fibonacci:
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
vaishnavibairagoni@Vaishnavis-Laptop AI AC % python3 assign1.py
```